

Last modified on

Sports Data Soccer API - Player Contract Feed (PE12)

Get contract related information for a player

Overview

The Soccer **Player Contract feed** provides information related to a player’s membership and his terms of contract like team ID, start date, end date, type of contract, along with basic player level information like first name, last name, nationality etc...

Required Compliance:

To comply with our API security and best practice logic for your API calls please refer to our **FAQs**. We also recommend that you take a look at our **API Overview and Authentication Guide**. You will learn the basics of how our API works and key concepts to get started.

How Often Should I Poll The Feed?

We have provided some guidelines to help ensure that you call the feed only as often as you need to. Make sure that you don't exceed the rate confirmed with us - this can vary depending on whether you make requests as a B2B or B2C outlet - if you need more information on this, contact your Stats Perform representative.

POLLING FREQUENCY
Once in 24 hours

Player Contract (PE12) Guide

USAGE	
GET player contract by person UUID (path parameter method)	/soccerdata/playercontract/{ou {queryParameters}
GET player contract by person UUID (prsn query parameter method)	/soccerdata/playercontract/{ou {queryParameters}
GET player contract by person legacy Opta ID (prsn query parameter method by urn)	/soccerdata/playercontract/{ou prsn={urn:perform:opta:person:
GET player contract by contestant UUID (ctst query parameter method)	/soccerdata/playercontract/{ou &{queryParameters}

USAGE

GET **player contract** by contestant legacy Opta ID (`ctst` query parameter method by `urn`) `/soccerdata/playercontract/{outletAuthKey}?ctst={urn:perform:opta:contestantId}`

If your request URL is as follows (does not contain this query criteria), the feed returns an error:
`/soccerdata/playercontract/{outletAuthKey}`

Query Parameters And Example Request Parameters

This feed makes use of the **Global query parameters** and you can use the following query parameters. You must query the `/playercontract` feed by a specific UUID of a person or contestant (team) - you can combine additional query parameters in a request. See also: [Using Query Parameters](#) and [Global query parameters](#) for a guide.

Legacy IDs: You can use the `prsn` and `ctst` parameter to filter by legacy IDs - only one ID is supported (you cannot pass multiple IDs in a comma-separated list).

How to use query parameters and make successful requests:

In our guides, we use HTTPS syntax for our example GET requests (rather than cURL, for example). All URLs are case-sensitive - this means that URL strings must contain the correct case and operators. You MUST format your requests correctly and include certain information; otherwise, an error will be returned. You can find information about the API, including the base URL (`{protocol}://{requestDomain}/{feedResource}`), in the [main guide](#) and [API Overview and Authorisation guide](#).

- Include the following information in the URL or header and make sure it is correct for your outlet: your unique and valid `{outletAuthKey}` this must be after the Outlet Authentication Key (and endpoint, if applicable). for `_rt={mode}` (operating mode) and `_fmt={dataFormat}` (response format), entity query parameter and UUID.
- Begin the query parameter string with `?` (question mark) - this must be after the Outlet Authentication Key (and endpoint, if applicable). To build up a query string, make sure that each query parameter is separated by the `&` (ampersand) operator. Values must be separated by the correct operator, for example `&` (multiple values, 'AND'), or - if a parameter supports it - `,` (multiple values, 'OR') or `!` ('NOT').
- Use the required `_` (underscore) for any parameters listed with this prefix.
- Remove any placeholder value curly braces `{ }` from your calls (we include these as an example only).
- If using the JSONP format (`_fmt=jsonp`) you MUST use it in combination with the callback parameter (`_clbk`) and a fixed value (or where you use a different value in a different instance, you must do this as little as possible). Be aware that common JavaScript frameworks, such as jQuery, can default to use randomly-generated function names - you must make sure that these are overridden and define a fixed function name instead.

Query parameter	Value and description
<code>prsn</code>	GET player contract by person UUID (<code>prsn</code> query parameter method) <pre>GET https://api.performfeeds.com/soccerdata/playercontract/{outletAuthKey}?prsn={personUuid}?_rt={mode}&_fmt={dataFormat}</pre>

Query parameter	Value and description
<code>{personUuid}</code> (path parameter)	GET player contract by person UUID (path parameter method) GET https://api.performfeeds.com/soccerdata/playercontract/{outletAuthKey}/{personUuid}?_rt={mode}&_fmt={dataFormat}
<code>ctst</code>	GET player contract by contestant UUID (<code>ctst</code> query parameter method) GET https://api.performfeeds.com/soccerdata/soccerdata/playercontract/{outletAuthKey}?ctst={contestantUuid}&_rt={mode}&_fmt={dataFormat}
<code>prsn</code> (leagcy opta id)	GET player contract > by person legacy Opta ID (<code>prsn</code> query parameter method by urn) GET https://api.performfeeds.com/soccerdata/playercontract/{outletAuthKey}?fx={urn:perform:opta:person:Opta person Id}&_rt={mode}&_fmt={dataFormat}
<code>ctst</code> (leagcy opta id)	GET player contract by contestant legacy Opta ID (<code>ctst</code> query parameter method by urn) GET https://api.performfeeds.com/soccerdata/playercontract/{outletAuthKey}?fx={urn:perform:opta:contestant:Opta contestant Id}&_rt={mode}&_fmt={dataFormat}

Sample Calls

These sample calls let you see an example URL and the response - click on the links below (hyperlinks open in a new window).

- [XML](#)
- [JSON](#)

Feed Output

Here, you can find an explanation of all possible response fields and data. This is XML but fields in the feed response are similar for JSON.

- ▶<playerContract> (XML only)
- ▶<person>
- ▶<membership>
- ▶<contract>